

	Network Administration <i>Deployment of an Internal DNS-Based Web Server</i>	2025/2026
---	---	------------------

Objectives: • Understand and implement DNS name resolution for internal services. • Deploy and configure a web server accessible through a DNS hostname, ensuring secure access via HTTPS
--

Background / Scenario

A web server is a network service that delivers dedicated content to users using the known protocol HTTP. Deploying an internal web server allows organizations to host applications and resources that are accessible within a private network. When combined with proper network configuration, such deployments provide a scalable and controlled environment for services' delivery, closely reflecting real-world network architectures.

This lab focuses on the deployment of a web server in a network environment, where a DNS service has already been configured (Lab0, Lab3, Lab4). The objective is to make the web server accessible to users using a DNS hostname. The objectives are:

- Deploy an internal web server: set up and configure a web server within an isolated network namespace (Lab0).
- Integrate with static networking: ensure web server accessibility using static IP addressing and existing routing/NAT configuration (Lab3).
- Verify web service availability: use client namespaces and diagnostic tools to confirm access to the web server via its DNS hostname (Lab4).

In summary, this lab assumes that the DNS is already configured and reachable at 192.168.30.2, while PC1 and PC2 successfully resolve DNS names. Specifically, the main objective is to deploy an internal web server, make it accessible from PC1 and PC2 using a DNS hostname, while validating end-to-end connectivity from DNS resolution to HTTP service access.

So, we are assuming the following:

- Network topology already exists (Lab0)
- Static routing + static NAT already work (Lab3)
- DNS server already works but uses lab.local (Lab4)
- **DNS is NOT yet configured for the Web server. We must solve the .local conflict properly**

Part 1: Verify network configuration

Step 1: Execute the Lab0's script

PCs and Gateway should already have static IPs:

- PC1 → 192.168.10.10/24
- PC2 → 192.168.20.10/24
- Gateway LAN → 192.168.10.1 / 192.168.20.1
- ISP → 209.165.200.225/27

Step 2: Test basic connectivity

#All LAN ↔ Gateway and Gateway ↔ ISP links should work.

```
sudo ip netns exec PC1 ping -c 3 192.168.10.1
```

```
sudo ip netns exec PC2 ping -c 3 192.168.20.1
```

```
sudo ip netns exec GATEWAY ping -c 3 209.165.200.225
```

Step 3: Execute Lab 3's script (SNAT+ Internet access)

#All LAN ↔ Gateway and Gateway ↔ ISP links should work.

```
sudo ip netns exec PC1 ping -c 3 192.168.10.1
```

```
sudo ip netns exec PC2 ping -c 3 192.168.20.1
```

```
sudo ip netns exec GATEWAY ping -c 3 209.165.200.225
```

Step 4: Verify the topology

Network	Purpose
192.168.10.0/24	LAN 1 (PC1)
192.168.20.0/24	LAN 2 (PC2)
192.168.30.0/24	DNS Network

Key devices, DNS is not in the same LAN, so traffic must be routed.

PC1 → 92.168.10.10

PC2 → 92.168.20.10

Gateway → Routes between all networks

DNS Server → 192.168.30.2

Step 5: Execute Lab 4's script (DNS)

Execute the DNS script allowing the creation of DNS server, connecting it to the Gateway, assigning IPs, creating zone files, starting BIND, and configuring PCs to use DNS. Running this script allows deploying the DNS lab quickly and reliably, ensuring that the network, DNS, and routing configurations are applied correctly every time.

Step 6: Test and verification

```
sudo ip netns exec PC1 ping -c 1 192.168.30.2
sudo ip netns exec PC2 ping -c 1 192.168.30.2
sudo ip netns exec PC1 dig PC2.lab.local +short
sudo ip netns exec PC1 dig -x 192.168.30.2 +short
sudo ip netns exec PC1 dig google.com +short
# You can eventually debug as follows. Silent success/OK means configuration is valid
sudo ip netns exec DNS named-checkconf
sudo ip netns exec DNS named-checkzone lab.local /etc/bind/db.lab.local
```

Until this point, the network namespaces for PC1, PC2, Gateway, and the ISP have been created and assigned static IPs, routing between LANs and the ISP has been configured, SNAT has been applied to allow Internet access from the LANs, and the DNS server is deployed on a separate network with proper routing in place, enabling hostname resolution across the internal networks.

The next part of the lab will be dedicated to deploying a web server within its own namespace and configuring HTTPS, allowing secure access to the web service from the LAN hosts using DNS-based hostname resolution.

Part 2: Install and configure Web Server

Step 1: Create and configure the web server

```
sudo ip netns add WEB
ip netns list
# Connect WEB to the GATEWAY
sudo ip link add veth-gw-web type veth peer name veth-web
sudo ip link set veth-gw-web netns GATEWAY
sudo ip link set veth-web netns WEB
# Assign IP addresses, WEB/GATEWAY
sudo ip netns exec WEB ip addr add 192.168.40.2/24 dev veth-web
sudo ip netns exec WEB ip link set veth-web up
sudo ip netns exec WEB ip link set lo up
sudo ip netns exec GATEWAY ip addr add 192.168.40.1/24 dev veth-gw-web
sudo ip netns exec GATEWAY ip link set veth-gw-web up
# Add routing, WEB default route and Client routes
sudo ip netns exec WEB ip route add default via 192.168.40.1
sudo ip netns exec PC1 ip route replace 192.168.40.0/24 via 192.168.10.1
sudo ip netns exec PC2 ip route replace 192.168.40.0/24 via 192.168.20.1
# Test connectivity
# Ping the gateway from WEB
sudo ip netns exec WEB ping -c 3 192.168.40.1
# Ping PC1 and PC2 from WEB
sudo ip netns exec WEB ping -c 3 192.168.10.10
sudo ip netns exec WEB ping -c 3 192.168.20.10
```

Step 2: Check and configure DNS in WEB

```
sudo ip netns exec WEB bash -c 'echo "nameserver 192.168.30.2" > /etc/resolv.conf'
sudo ip netns exec WEB bash -c 'echo "nameserver 8.8.8.8" >> /etc/resolv.conf'

#Your /etc/resolv.conf in WEB should now look like this:
# nameserver 192.168.30.2
# nameserver 8.8.8.8
# This means that:


- 192.168.30.2: points to your lab DNS server
- 8.8.8.8: Google's public DNS for external domains (like packages.nrl.navy.mil)

```

Step 3: Test that DNS is working

```
# Should return valid IP addresses, DNS resolution is working inside the WEB namespace.
sudo ip netns exec WEB dig google.com
```

Step 4: Test that NAT is working

```
# If this fails, NAT is not forwarding traffic
sudo ip netns exec WEB ping 8.8.8.8
```

Step 5: Test that NAT is working

```
# Identify the correct GATEWAY WAN interface. All WEB traffic to go out to the ISP
namespace via NAT.
sudo ip netns exec GATEWAY iptables -t nat -A POSTROUTING -s 192.168.40.0/24 -o
veth3 -j MASQUERADE
# Ensure routing is correct in ISP. Tells ISP where to send packets back to WEB via
GATEWAY.
sudo ip netns exec ISP ip route add 192.168.40.0/24 via 209.165.200.226
```

Step 6: Test internet connectivity¹

```
# Must successfully work
sudo ip netns exec WEB ping -c 3 8.8.8.8
sudo ip netns exec WEB curl http://example.com
```

Step 7: Install Apache Web Server

```
# Update and install apache
sudo ip netns exec WEB apt update
sudo ip netns exec WEB apt install -y apache2
# Fix runtime directories (required for namespace)
sudo ip netns exec WEB mkdir -p /run/apache2 /var/log/apache2
sudo ip netns exec WEB chown www-data:www-data /run/apache2 /var/log/apache2
# Start Apache
sudo ip netns exec WEB bash -c 'source /etc/apache2/envvars && /usr/sbin/apache2 -k start -
DFOREGROUND &'
# Verify Apache is running, should show Apache listening on port 80
sudo ip netns exec WEB ps aux | grep apache2
sudo ip netns exec WEB ss -tulnp | grep :80
```

Step 8: Create and basic tests of a web page

```
# Create basic testing page
sudo ip netns exec WEB bash -c 'echo "<h1>Web Server is running</h1>" >
/var/www/html/index.html'
# Test web page, from WEB first
sudo ip netns exec WEB curl http://127.0.0.1
# From PC1 by IP
sudo ip netns exec PC1 curl http://192.168.40.2
# Should return <h1>Web Server is running</h1>
```

Step 9: Test web page from PC1 by DNS

```
sudo ip netns exec PC1 curl http://web.lab.test
```

¹ Why dig works but apt update fails: dig google.com uses UDP port 53 so works through your DNS server, apt update uses TCP port 80/443 to download .deb packages

The abovementioned commands will return “curl: (6) Could not resolve host: web.lab.test”. This does not mean the web server is down, the error is about DNS resolution, not connectivity. It is worth noting that curl first asks the system’s DNS server to resolve web.lab.test. PC1 sends the query to whatever is in /etc/resolv.conf, if the DNS server does not have a record for web.lab.test, it cannot return an IP. As a result, curl reports “Could not resolve host”. So, DNS must include the new host, since in Lab 4, the original DNS only had PC1 and PC2. Adding WEB, means updating the DNS zone file.

Step 10: Add Web Server to DNS

```
# Stop DNS server temporarily
sudo ip netns exec DNS systemctl stop bind9
# Rename zone file if still using .local (fix conflict)
sudo ip netns exec DNS mv /etc/bind/db.lab.local /etc/bind/db.lab.test
# Update zone declaration, as follows
sudo ip netns exec DNS vi /etc/bind/named.conf.local
zone "lab.test" {
    type master;
    file "/etc/bind/db.lab.test";
};
# Update zone file content
$TTL 604800
@ IN SOA ns.lab.test. admin.lab.test. (
    20260101
    604800
    86400
    2419200
    604800 )
;
@ IN NS ns.lab.test.

ns IN A 192.168.30.2
pc1 IN A 192.168.10.10
pc2 IN A 192.168.20.10
web IN A 192.168.40.2
```

Step 11: Verify DNS

```
# Verify DNS is listening
sudo ip netns exec DNS ss -ulnp | grep :53
```

Step 12: Add Web Server to DNS

```
# Final test
sudo ip netns exec PC1 dig web.lab.test @192.168.30.2 +short
sudo ip netns exec PC1 curl http://web.lab.test2
```

² Killing the old named process and starting a fresh one inside the DNS namespace sometime fixed any problem. Always stop/kill named before restarting, especially in namespaces.

```
sudo ip netns exec DNS pkill named || true
sudo ip netns exec DNS named -g -u root -c /etc/bind/named.conf &
```

Step 13: Create a Bash script to automate the Web server lab

Create a Bash script (`enable_webserver.sh`) to automate the deployment of an internal web server in the lab environment, assuming that the DNS service has already been configured in Lab 4 (including Lab 0 and Lab 3). The script only sets up the web server within an isolated network namespace, assigns a static IP address, connects it to the Gateway, and installs and starts the server. It ensures that the web server is fully integrated with the existing static networking, routing, and NAT configuration and that it can be accessed by client namespaces using its DNS hostname. Running this script enables consistent and repeatable web server deployment and allows verifying web service availability.

Step 14: Bash script to clean and reset the Server lab

Create a Bash script (`cleanup_webserver.sh`) to safely remove the web server setup from the lab environment. This script stops and deletes the web server network namespace, removes associated virtual interfaces, and clears any static IP, routing, and NAT rules related to the web server. It ensures that the cleanup process does not affect the previously configured DNS service (Lab 4) or other lab components (from Lab 0 and Lab 3). Executing this script allows the lab environment to be reset to a clean state, enabling repeated experimentation, troubleshooting, and redeployment of the web server in a controlled and consistent manner.

Part 3: Enable HTTPS on the WEB Server

So far, you have the Web namespace already running and has Apache installed, DNS is configured so web.lab.test resolves to the WEB server IP 192.168.40.2. In the rest of this lab, we will enable HTTPS for the WEB server and conduct related testing and verification:

- Enables SSL and security headers
- Creates a self-signed certificate
- Configures Apache for HTTPS on port 443
- Tests locally inside WEB and from PC1
- Leaves Apache running with HTTPS only

Step 1: Install SSL modules and OpenSSL

```
# For script safety. If any command fails, the script stops immediately.
set -e
# Updates the package list inside the WEB namespace.
sudo ip netns exec WEB apt update
# Installs OpenSSL (used to create certificates) and Apache if not installed.
sudo ip netns exec WEB apt install -y openssl apache2
# Enables Apache modules. Apache can now handle SSL/TLS connections.
sudo ip netns exec WEB a2enmod ssl
sudo ip netns exec WEB a2enmod headers
```

Step 2: Generate a self-signed certificate

```
# Create directory for SSL files. Stores private key (.key) and certificate (.crt)
sudo ip netns exec WEB mkdir -p /etc/apache2/ssl

sudo ip netns exec WEB openssl req -x509 -nodes -days 365 \
  -newkey rsa:2048 \
  -keyout /etc/apache2/ssl/$DOMAIN.key \
  -out /etc/apache2/ssl/$DOMAIN.crt \
  -subj "/C=US/ST=State/L=City/O=Lab/OU=IT/CN=$DOMAIN"
```

Explanation

Option	Meaning
req -x509	Create a self-signed certificate
-nodes	No password (Apache can start automatically)
-days 365	Valid for 1 year
rsa:2048	Secure key length
CN=web.lab.test	Must match Apache ServerName

Step 3: Create HTTPS Virtual Host configuration

```
# Key concepts: *:443 → HTTPS port, ServerName → domain name, SSLEngine on →
activates TLS, HSTS → forces browsers to always use HTTPS
sudo ip netns exec WEB bash -c 'cat > /etc/apache2/sites-available/web-ssl.conf <<EOF
<VirtualHost *:443>
    ServerName web.lab.test
    DocumentRoot /var/www/html

    SSLEngine on
    SSLCertificateFile /etc/apache2/ssl/web.lab.test.crt
    SSLCertificateKeyFile /etc/apache2/ssl/web.lab.test.key

    <Directory /var/www/html>
        AllowOverride All
        Require all granted
    </Directory>

    Header always set Strict-Transport-Security "max-age=63072000; includeSubDomains;
preload"
</VirtualHost>
EOF'

# Check the file web-ssl.conf
sudo ip netns exec WEB vi /etc/apache2/sites-available/web-ssl.conf
```

Step 4: Enable HTTPS site and restart Apache

```
# Restart Apache in a lab namespace
sudo ip netns exec WEB a2ensite web-ssl.conf
sudo ip netns exec WEB bash -c 'source /etc/apache2/envvars && /usr/sbin/apache2 -k restart'
```

Step 5: Create a test HTTPS page

```
# Simple HTML page to confirm everything works.
sudo ip netns exec WEB bash -c 'echo "<h1>Web Server is running (HTTPS)</h1>" >
/var/www/html/index.html'
# Confirms Apache is listening on 443 and SSL is functional
sudo ip netns exec WEB curl -k https://127.0.0.1
# Confirms routing is correct, firewall allows HTTPS, and end-to-end encryption works
sudo ip netns exec PC1 curl -k https://192.168.40.2
sudo ip netns exec PC1 curl -k https://web.lab.test
```

Step 6: Bash script to enable HTTPS

By using the commands above, an HTTPS-enabled Apache server secured with SSL/TLS is now running inside a Linux namespace and has been tested both locally and remotely. Develop a script that enables HTTPS and verifies that the HTTPS page is accessible from the PCs, thereby automating the process and avoiding manual input each time.

Step 7: Bash script to clean and reset HTTPS

Develop a cleaning script that removes HTTPS cleanly from the Apache web server running inside the WEB namespace, without uninstalling Apache, and restores the server to HTTP-only mode. It then verifies that HTTP still works both locally and from a client PC.

Step 8: Bash script for external HTTPS Exposure (Bonus)

In the current setup, the web server is internal, so external access requires explicit configuration on the Gateway. This means:

- The web server is running on a private IP address
- (for example: 192.168.40.2)
- It exists inside the lab network, in a Linux network namespace
- It is not directly visible from outside the network (Internet / ISP)
- Private IP addresses (192.168.x.x) cannot be reached directly from external networks.

The Gateway is the only device that:

- Connects the internal network to the outside
- Has a public IP address
- Controls traffic going in and out

To allow external access, the Gateway must be told the following: *when traffic arrives from outside on a certain port, forward it to the internal web server.*

Prepare a new script which allows exposing the HTTPS web server to an external network using Destination Network Address Translation (DNAT) on the Gateway. This demonstrates how inbound access to private services is enabled in real-world networks while preserving internal IP addressing and security boundaries.

Deliverables (To be uploaded on the Platform)

In this lab, you designed and configured a Linux-based web server environment using network namespaces, building on the DNS infrastructure established in the previous lab. A web server is deployed within a dedicated network namespace and connected to the Gateway namespace. The LAN hosts (PC1 and PC2), as introduced in Lab 0, are configured to access the web server using its DNS hostname, relying on the DNS service configured earlier for name resolution. This setup allows you to validate web service deployment, network connectivity, and DNS-based access in an isolated and controlled network environment. Furthermore, HTTPS has been enabled on the web server, providing encrypted communication between clients and the server. A self-signed SSL certificate is generated within the web server namespace, and the Apache server is configured to serve the web page securely over port 443. This allows you to validate secure web service deployment, network connectivity, and DNS-based access in an isolated and controlled network environment, ensuring that clients can successfully reach the web server both over HTTP and HTTPS.

The main report document (Word/PDF, 10 pages max) must include the following:

- Web Server Configuration
 - Namespace and network setup: IP address of the web server and Gateway, static routing/NAT configuration, and routing tables showing client-to-web server paths.
 - Explanation: how Linux network namespaces are used to emulate isolated servers and hosts in a realistic network environment.
- Verification and Testing
 - Web service functionality: access the web server from PC1 and PC2 using its DNS hostname.
 - DNS and routing verification: confirm hostname resolution and check routing tables in each namespace using `ip route`.
 - Traffic verification: use `tcpdump` to capture and analyze requests and responses exchanged between the clients and the web server.
 - HTTPS verification: confirm that the web server is accessible securely over HTTPS from the clients and that encryption is functioning correctly.
- Conclusion
 - Highlight that the web server is correctly deployed and accessible to both LAN hosts using its DNS hostname. Discuss the role of DNS in enabling service discovery, the importance of proper routing and networking, and the benefits of using Linux network namespaces to emulate real-world web service deployments in a controlled lab environment. Also, verify that the web server can be accessed securely over HTTPS, demonstrating SSL/TLS encryption in the lab environment.

A single compressed file named in the format “lastname.firstname-Lab5.zip”, must be uploaded on the platform (see deadlines) and must include:

- The main report document (Word or PDF)
- All developed Bash scripts